

# Labo 00 — Becommentarieerde antwoorden

*Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of de valkuil, een voorbeeld dat je aan de terminal kunt verifiëren.*

## Vraag 1 — Een Linux-binary draait overal... behalve op Windows

**Antwoord.** Een binary vraagt niets aan "Ubuntu" of "Alpine": hij vraagt alles aan de **kernel**, via de system calls (`open`, `read`, `fork`...). Die calls zijn identiek op elke Linux-machine: de distributie levert alleen de userland eromheen. De Windows-kernel biedt andere, incompatibele calls: de binary heeft er geen gesprekspartner.

**Waarom.** De interface kernel ↔ programma's is stabiel en gestandaardiseerd (Linux verbiedt zichzelf ze te breken). Alles wat distributies onderscheidt — pakketbeheerder, bibliotheekversies, configuratie — leeft *boven* die interface.

**Nuance.** "Identiek" veronderstelt dat de nodige dynamische bibliotheken aanwezig zijn (de `libc` bijvoorbeeld, verschillend tussen Debian en Alpine — je botst erop in labo 05). En WSL "vertaalt" niet: WSL 2 draait een **echte** Linux-kernel in een VM.

### Voorbeeld.

```
uname -r          # 6.6.87.2-microsoft-standard-WSL2: een echte Linux-kernel, getekend
Microsoft
uname -m          # x86_64: de architectuur, de andere compatibiliteitsvoorwaarde
```

## Vraag 2 — De wees geadopteerd door PID 1

**Antwoord.** De shell (ouder van de `sleep`) stierf met de terminal. De kernel laat een proces nooit zonder ouder: de wees wordt **geadopteerd** door PID 1 (`systemd`), vandaar PPID = 1. `sleep` draait gewoon verder.

**Waarom.** De ouder heeft een precieze rol: wanneer een kind sterft, leest hij zijn exitcode (anders blijft het kind in "zombie"-toestand). Er moet dus altijd een voogd van laatste toevlucht zijn — een van de verantwoordelijkheden van PID 1.

**Nuance.** Precies daarom is de PID 1 *van een container* een ernstig onderwerp (labo 03): jouw applicatie erft daar die voogdijrol zonder het te weten, en een PID 1 die zijn zombies niet begraaft of geen reserve-ouder heeft, verandert het gedrag van de container.

### Voorbeeld.

```
sleep 300 &
ps -o pid,ppid,cmd | grep [s]leep    # 2419 2363 sleep 300 (PPID = jouw bash)
# sluit de terminal, open een nieuwe:
ps -ef | grep [s]sleep                # ubuntu 2419 1 ... sleep 300 (PPID = 1)
```

## Vraag 3 — Permission denied, code 126

**Antwoord.** Het bestand heeft de uitvoerbit `x` niet. Bevestiging: `ls -l deploy.sh` (je leest `-rw-r--r--`, geen `x`). Oplossing: `chmod +x deploy.sh`. Omweg zonder iets te verhelpen: `bash`

`deploy.sh` — dan wordt `bash` gestart (zelf wél uitvoerbaar), en is het script slechts een argument dat gelezen wordt.

**Waarom.** `./deploy.sh` starten vraagt de kernel om **dat bestand uit te voeren**; de kernel controleert de `x`-bit en weigert. Code 126 is de shellconventie: "gevonden, maar niet uitvoerbaar" — te onderscheiden van 127, "niet gevonden".

**Nuance.** Een bestand aangemaakt door een editor of gedownload wordt geboren als `rw-`: uitvoering is een recht dat je bewust toevoegt. In een Dockerfile (labo 04) faalt de `COPY` van een script gevolgd door `RUN ./script.sh` op dezelfde manier als de `x`-bit in je Git-repository ontbrak.

#### Voorbeeld.

```
printf '#!/bin/bash\nnecho hallo\n' > deploy.sh
./deploy.sh ; echo $?      # bash: ./deploy.sh: Permission denied ; 126
chmod +x deploy.sh
./deploy.sh                # hallo
```

### Vraag 4 — Shellvariabele vs omgevingsvariabele

**Antwoord.** Regel 1: `1:` (leeg). Regel 2: `2: hallo`. Vóór `export` is `MSG` alleen een variabele **van de huidige shell**; het kind `bash -c` wordt geboren met de geërfde omgeving, waarin `MSG` niet bestaat. Na `export` zit `MSG` in de omgeving, en elk kind krijgt er een kopie van.

**Waarom.** Bij zijn creatie krijgt een proces een **kopie** van de omgeving van zijn ouder, nooit een referentie: het is een eenrichtingserfenis, bevroren op het moment van de start.

**Nuance.** De **enkele** aanhalingstekens van `'echo 1: $MSG'` zijn essentieel: ze beletten jouw shell om `$MSG` te vervangen vóór het kind start. Met dubbele aanhalingstekens zouden beide regels `hallo` tonen... maar dan had de ouder de substitutie gedaan, niet het kind. Belangrijk gevolg: de omgeving van een **al gestart** proces wijzigen is onmogelijk — vandaar de `podman run -e` die bij de start wordt vastgelegd (labo 08).

#### Voorbeeld.

```
MSG=hallo
env | grep MSG      # niets
export MSG
env | grep MSG      # MSG=hallo
```

### Vraag 5 — Code 137

**Antwoord.**  $137 = 128 + 9$ : het proces stierf door signaal nummer 9, `SIGKILL`. Niemand gaf het ook maar de minste kans om netjes te stoppen. Die code is beroemd omdat het die van een neergeschoten container is: door `docker kill`, door een `docker stop` die na zijn gratieperiode geen antwoord kreeg, of door de **OOM killer** van de kernel bij geheugengebrek.

**Waarom.** De shell codeert de dood door signaal als `128 + nummer`, om ze te onderscheiden van een vrijwillige `exit n`. `SIGKILL` wordt nooit aan het proces afgeleverd: de kernel wist het rechtstreeks, zonder opruimcode uit te voeren.

**Nuance.** Een 137 diagnosticeren betekent dus zoeken **wie** de 9 stuurde: een mens, een orchestrator... of de kernel. `dmesg | grep -i "out of memory"` beslecht het OOM-geval. Je maakt die diagnose op echte containers in labo 03.

## Voorbeeld.

```
bash -c 'kill -9 $$' ; echo $?    # 137 ($$ = de PID van de kind-bash zelf)
sleep 300 & kill -9 $! ; wait $! ; echo $?    # ook 137
```

## Vraag 6 — `kill` beleefd, `kill -9` brutaal

**Antwoord.** `kill` (dus `SIGTERM`) is een **verzoek**: het proces ontvangt het, kan zijn stopcode uitvoeren — buffers legen, transacties afronden, verbindingen sluiten — en dan eindigen. `kill -9` (`SIGKILL`) wordt niet aan het proces afgeleverd: de kernel wist het onmiddellijk. Een databank verliest dan alles wat nog niet op schijf stond en moet bij de herstart haar journaal herspelen — of zelfs beschadigde bestanden herstellen.

**Waarom.** Dit is exact het protocol van `docker stop`: `SIGTERM` naar PID 1 van de container, gratieperiode (standaard 10 s), dan `SIGKILL` als het proces niet gehoorzaamde. Een applicatie die `SIGTERM` negeert wordt dus **altijd** brutaal gedood na de termijn.

**Nuance.** `kill -9` heeft zijn plaats: een geblokkeerd proces dat `SIGTERM` werkelijk negeert. De juiste reflex is de escalatie — `kill`, wachten, dan pas `kill -9` — nooit omgekeerd. Merk ook op dat `SIGKILL` niet opgevangen noch genegeerd kan worden: het is het enige gegarandeerde redmiddel.

## Voorbeeld.

```
sleep 300 &
kill %1      # SIGTERM: de job toont "Terminated"
sleep 300 &
kill -9 %1   # SIGKILL: de job toont "Killed"
```

## Vraag 7 — `-rw-r-----` root shadow lezen

**Antwoord.** De negen bits vallen uiteen in drie tripletten: eigenaar `rw-`, groep `r--`, anderen `--`. Jouw gebruiker is noch `root` (eigenaar) noch lid van de groep `shadow`: hij valt onder "anderen", die **geen enkel** recht hebben — vandaar de weigering. `sudo cat` voert `cat` uit met UID 0, en op root past de kernel de permissiecontroles niet toe. Zonder `sudo` kunnen alleen root en de leden van de groep `shadow` (lezend) het bestand lezen.

**Waarom.** Bij elke `open` vergelijkt de kernel de UID/GID van het aanroepende **proces** met de bits van het bestand: eerst eigenaar, anders groep, anders "anderen". Het eerste toepasselijke triplet is het enige dat wordt toegepast.

**Nuance.** `/etc/shadow` bevat de wachtwoordhashes — hét canonieke voorbeeldbestand. Merk op dat de regel "eerste toepasselijke triplet" kan verrassen: een bestand `----rw-rw-` zou onleesbaar zijn... voor zijn eigen eigenaar.

## Voorbeeld.

```
id                # uid=1000(ubuntu) ...: geen root, geen groep shadow
cat /etc/shadow   # Permission denied, code 1
sudo head -n 1 /etc/shadow # root:!:20501:0:99999:7:::
```

## Vraag 8 — Twee stromen, twee bestanden

**Antwoord.** Het scherm toont **niets**. `resultaat.txt` bevat de regel van het succes (`/etc/hostname`); `fouten.txt` bevat de melding `ls: cannot access '/onbekende-datum': No such file or directory`. Met `> resultaat.txt 2>&1` zouden beide regels in `resultaat.txt` belanden en zou `fouten.txt` niet worden aangemaakt.

**Waarom.** `ls` schrijft zijn resultaten naar **stdout** (stroom 1) en zijn klachten naar **stderr** (stroom 2). `>` leidt alleen stroom 1 om, `2>` alleen stroom 2; `2>&1` betekent "laat stroom 2 wijzen naar waar stroom 1 *nu* naartoe wijst".

**Nuance.** De volgorde telt: `2>&1 > resultaat.txt` zou de fouten... naar het scherm sturen (stroom 2 wordt aangesloten op de oude stroom 1, vóór de redirection). Deze scheiding van stromen is wat `podman logs` straks toelaat om je de fouten én de normale uitvoer van een container te tonen (labo 03).

### Voorbeeld.

```
ls /etc/hostname /onbekende-datum > resultaat.txt 2> fouten.txt
cat resultaat.txt      # /etc/hostname
cat fouten.txt         # ls: cannot access '/onbekende-datum': No such file or directory
```

## Vraag 9 — 127.0.0.1 vs 0.0.0.0

**Antwoord.** Redis luistert op `127.0.0.1:6379`: alleen de *loopback*-interface, dus **alleen bereikbaar vanaf de machine zelf**. Java luistert op `0.0.0.0:8080`: alle interfaces, dus bereikbaar vanaf het netwerk. Vanaf een andere machine antwoordt alleen de Java-dienst.

**Waarom.** Het luisteradres is een filter: de kernel geeft het proces alleen de verbindingen door die op dat adres aankwamen. `0.0.0.0` betekent "alle adressen van de machine".

**Nuance.** Dit is een beveiligingsgrens van eerste orde: een databank die lokaal luistert, is niet aanvalbaar vanaf het netwerk. In labo 07 zul je zien dat `podman run -p 8080:80` standaard op `0.0.0.0` publiceert — en dat `-p 127.0.0.1:8080:80` bewust beperkt. Wie deze twee `ss`-regels begrijpt, begrijpt `-p` al.

### Voorbeeld.

```
python3 -m http.server 8080 --bind 127.0.0.1 &
ss -tlnp | grep 8080      # LISTEN ... 127.0.0.1:8080 ... ("python3",pid=...)
kill %1
```

## Vraag 10 — Poort 80 geweigerd

**Antwoord.** Poorten **onder 1024** (zogenaamd *geprivilegieerd*) kunnen alleen door root (UID 0) worden geopend. Jouw proces, UID 1000, krijgt de `bind` op poort 80 geweigerd; 8080 ligt boven de drempel en is dus vrij. Historisch garandeerde de regel dat op een gedeelde machine een "officiële" dienst (poort 25, 80...) niet door een gewone gebruiker kon worden nagebootst. Gevolg: Podman rootless, een gewoon proces met jouw UID, kan geen `-p 80:80` publiceren — je publiceert `-p 8080:80` in de plaats.

**Waarom.** De controle gebeurt door de kernel op het moment van de system call `bind`, op basis van de effectieve UID (preciezer: van een *capability* die root bezit).

**Nuance.** De drempel is instelbaar ( `sysctl net.ipv4.ip_unprivileged_port_start` ), en echte productiewebserver draaien achter een load balancer die zelf poort 80 bezit. De foutmelding van Podman ( `pasta failed ... Permission denied` ) komt terug in labo 07.

### Voorbeeld.

```
python3 -m http.server 80
# PermissionError: [Errno 13] Permission denied
python3 -m http.server 8080 & # werkt
kill %1
```

## Vraag 11 — `/proc`, de nepmap

**Antwoord.** `/proc` is een **virtueel bestandssysteem** (type `proc`), gemount op `/proc`, waarvan de inhoud op geen enkele schijf bestaat: elke lezing wordt ter plekke door de kernel gefabriceerd uit zijn interne toestand. De numerieke mappen zijn de levende processen; de andere bestanden beschrijven het systeem. Gebruiksvoorbeelden: `/proc/<pid>/environ` (de echte omgeving van een proces), `/proc/meminfo` (het geheugen), `/proc/self/uid_map` (de UID-toewijzingen — het bewijs van rootless in labo 01).

**Waarom.** "Alles is een bestand": de toestand van de kernel als bestanden tonen laat toe om `cat`, `grep` en `ls` als beheergereedschap te gebruiken, zonder aparte API. `ps` is slechts een verpakking van `/proc`.

**Nuance.** Daarom krijgt een container bij zijn creatie ook **zijn eigen** `/proc` gemount: anders zou hij alle processen van de host zien. Wanneer `ps` "liegt" in een container, is dat omdat zijn `/proc` geïsoleerd is — niet omdat de processen verdwenen zijn.

### Voorbeeld.

```
findmnt -t proc # /proc proc proc rw,relatime
df -h /proc 2>/dev/null # geen schijf gekoppeld
tr '\0' '\n' < /proc/self/environ | head -3 # de omgeving... van deze cat
```

## Vraag 12 — `command not found`, code 127

**Antwoord.** De shell doorliep, in volgorde, elke map van de variabele `PATH` op zoek naar een uitvoerbaar bestand `mijntool`; `~/tools` staat er niet in, de zoektocht faalt, code 127. Onmiddellijk: starten via expliciet pad, `~/tools/mijntool`. Duurzaam: (1) de map toevoegen aan het `PATH` in `~/.bashrc` ( `export PATH="$HOME/tools:$PATH"` ), of (2) de tool kopiëren/linken naar een map die er al in staat, zoals `~/local/bin` of `/usr/local/bin`.

**Waarom.** Het `PATH` is het enige mechanisme om "kale" commando's op te lossen. De huidige map hoort er bewust niet bij: een vervalste `ls` neergelegd in `/tmp` mag niet worden uitgevoerd omdat jij toevallig `cd /tmp` deed.

**Nuance.** 127 (niet gevonden) en 126 (gevonden maar niet uitvoerbaar) zijn twee verschillende diagnoses. In een container heeft de fout `exec: "mijntool": executable file not found in $PATH` exact dezelfde oorzaak — het `PATH` van de image (labo 04).

### Voorbeeld.

```
mkdir -p ~/tools && printf '#!/bin/bash\nnecho ok\n' > ~/tools/mijntool && chmod +x  
~/tools/mijntool  
mijntool                # command not found ; echo $? → 127  
export PATH="$HOME/tools:$PATH"  
mijntool                # ok
```

### Vraag 13 — Waarom 12-factor van de omgeving houdt

**Antwoord.** Omdat de omgeving vasthangt aan het **proces**, niet aan de machine: ze wordt bij de start vastgelegd, automatisch geërfd, en verdwijnt met het proces. Voor wegwerpbare en herstartbare applicaties geeft dat een configuratie die (1) van buitenaf te injecteren is zonder code of geleverde bestanden te wijzigen, (2) per instantie kan verschillen — twee processen naast elkaar met twee configuraties, (3) geen resttoestand achterlaat: herstarten met andere waarden volstaat, niets op te ruimen.

**Waarom.** De overerving ouder → kind doet al het werk: wie start (de shell, systemd, straks de containerengine) bereidt het woordenboek voor, de applicatie leest alleen. Hetzelfde artefact — JAR of image — gaat ongewijzigd van omgeving naar omgeving; alleen de set variabelen verandert.

**Nuance.** De onveranderlijkheid van de erfenis is ook haar limiet: een variabele wijzigen betekent het proces **herstarten**. Voor een container (wegwerpbaar bij ontwerp) is dat geen probleem, maar wel een rouwproces voor wie hoopte om al draaiend te herconfigureren. Geheimen verdienen straks beter dan variabelen die zichtbaar zijn in `/proc/<pid>/environ` (labo 08).

### Voorbeeld.

```
SERVER_PORT=9090 java -jar app.jar    # dezelfde JAR, andere poort – niets werd gewijzigd  
# dat wordt, woord voor woord:  
# podman run -e SERVER_PORT=9090 mijn-api
```